# Diamond Prices Linear Regression Project

Arden Chen, Alison Gu, Andrew Machon

## PART - 1

Data Description and Descriptive Statistics

Your first objective is to understand all the variables in the Diamonds dataset from Kaggle and explore their relationships: Here's the link. You are expected to include all available variables in your analysis, not just a subset. This means incorporating all categorical and numerical variables present in the dataset. Make sure your sample contains at least 1000 observations.

```
diamondsdata <- read.csv("DiamondsPrices2022.csv")
head(diamondsdata)
```

```
  X carat       cut color clarity depth table price    x    y    z
1 1  0.23     Ideal     E     SI2  61.5    55   326 3.95 3.98 2.43
2 2  0.21   Premium     E     SI1  59.8    61   326 3.89 3.84 2.31
3 3  0.23      Good     E     VS1  56.9    65   327 4.05 4.07 2.31
4 4  0.29   Premium     I     VS2  62.4    58   334 4.20 4.23 2.63
5 5  0.31      Good     J     SI2  63.3    58   335 4.34 4.35 2.75
6 6  0.24 Very Good     J    VVS2  62.8    57   336 3.94 3.96 2.48
```

1. Select a random sample. 2 points

```
library(ggplot2)
set.seed(12345)
diamonds_sample <- diamonds[sample(nrow(diamonds), 1000), ]
```

2. Describe all the variables (call summary function on the dataset, see the structure, create histograms for continuous random variable, comment on their distribution, bar plots for categorical random variable). 10 points

```
summary(diamonds_sample)
```

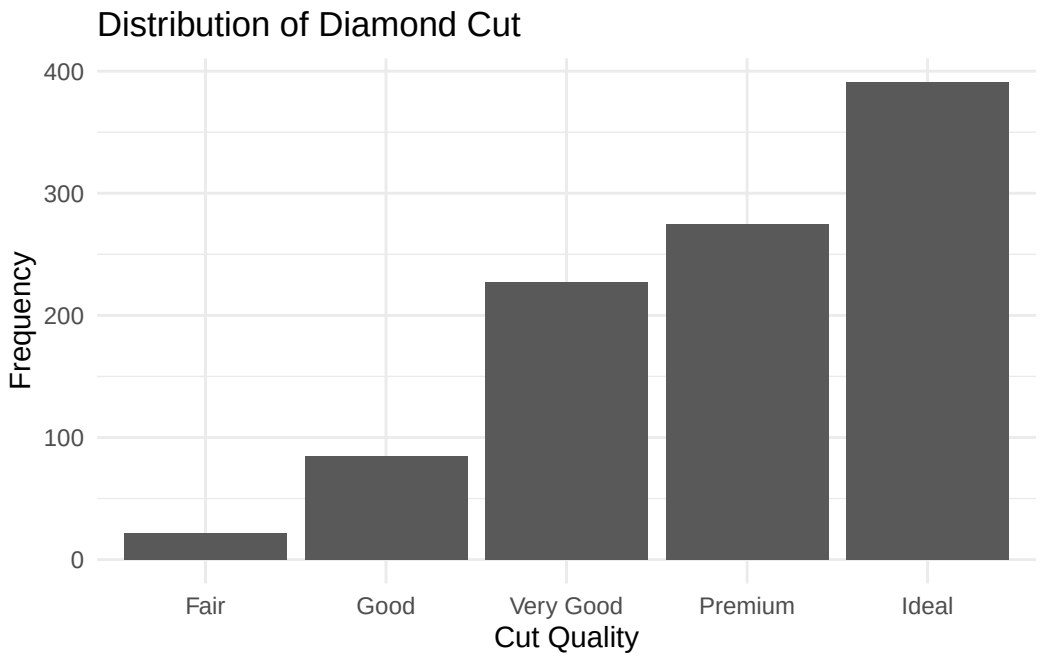
```
     carat               cut        color       clarity        depth      
 Min.   :0.2300   Fair     : 22   D:130   SI1    :238   Min.   :56.50  
 1st Qu.:0.4000   Good     : 85   E:193   VS2    :233   1st Qu.:61.00  
 Median :0.7000   Very Good:227   F:174   SI2    :165   Median :61.80  
 Mean   :0.7915   Premium  :275   G:208   VS1    :141   Mean   :61.67  
 3rd Qu.:1.0400   Ideal    :391   H:160   VVS2   : 97   3rd Qu.:62.50  
 Max.   :3.0000                   I: 97   VVS1   : 74   Max.   :68.60  
                                  J: 38   (Other): 52                  
     table           price                x                y         
 Min.   :52.00   Min.   :  357.0   Min.   :3.900   Min.   :3.840  
 1st Qu.:56.00   1st Qu.:  922.5   1st Qu.:4.720   1st Qu.:4.720  
 Median :57.00   Median : 2359.5   Median :5.660   Median :5.670  
 Mean   :57.59   Mean   : 3889.3   Mean   :5.714   Mean   :5.715  
 3rd Qu.:59.00   3rd Qu.: 5304.2   3rd Qu.:6.540   3rd Qu.:6.550  
 Max.   :68.00   Max.   :18700.0   Max.   :9.040   Max.   :8.980  

       z        
 Min.   :2.380  
 1st Qu.:2.910  
 Median :3.520  
 Mean   :3.524  
 3rd Qu.:4.030  
 Max.   :5.970  
```

```
str(diamonds_sample)
```

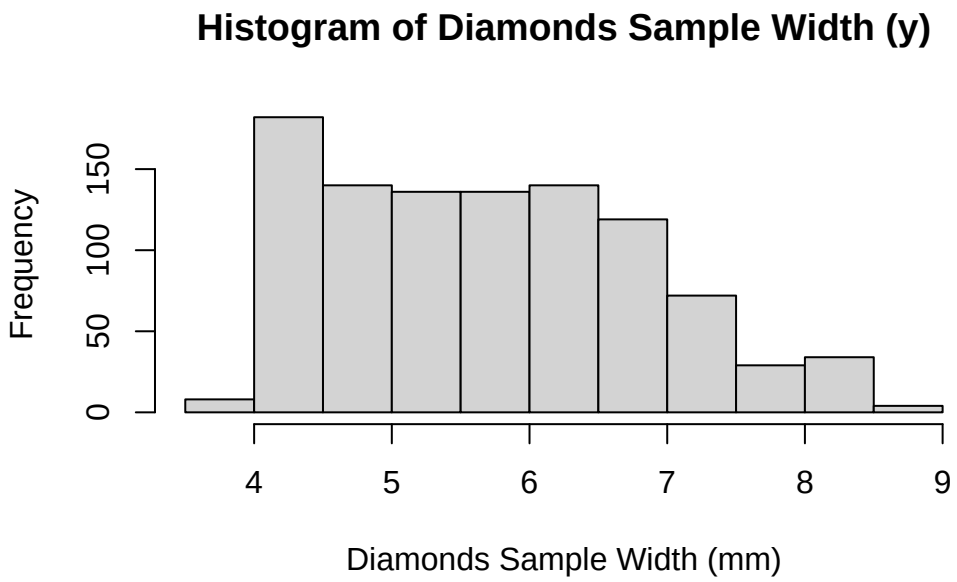
```
tibble [1,000 x 10] (S3: tbl_df/tbl/data.frame)
 $ carat  : num [1:1000] 1.01 0.41 1.26 0.3 0.34 2 1.16 0.54 1.51 0.31 ...
 $ cut    : Ord.factor w/ 5 levels "Fair"<"Good"<..: 5 3 5 2 5 3 5 4 4 4 ...
 $ color  : Ord.factor w/ 7 levels "D"<"E"<"F"<"G"<..: 5 4 6 5 3 3 4 2 4 4 ...
 $ clarity: Ord.factor w/ 8 levels "I1"<"SI2"<"SI1"<..: 3 4 2 3 4 3 7 4 5 6 ...
 $ depth  : num [1:1000] 62.1 61.7 59.6 63.7 62.1 63.3 62 60.7 62.6 62.9 ...
 $ table  : num [1:1000] 56 61 57 57 56 56 57 56 60 58 ...
 $ price  : int [1:1000] 4886 827 4704 554 726 15851 9526 1760 14375 907 ...
 $ x      : num [1:1000] 6.38 4.7 7.04 4.28 4.48 8 6.72 5.32 7.19 4.3 ...
 $ y      : num [1:1000] 6.47 4.74 7.01 4.26 4.51 7.93 6.74 5.29 7.28 4.28 ...
 $ z      : num [1:1000] 3.99 2.91 4.19 2.72 2.79 5.04 4.17 3.22 4.53 2.7 ...
```

```r
ggplot(diamonds_sample, aes(x = cut)) +
  geom_bar() +
  labs(title = "Distribution of Diamond Cut",
       x = "Cut Quality",
       y = "Frequency") +
  theme_minimal()
```



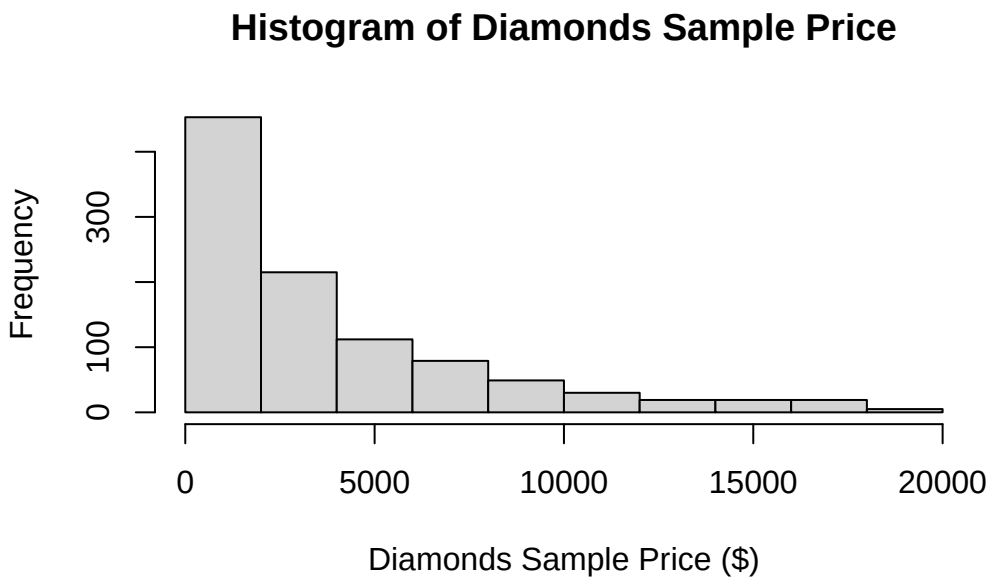

The ideal cut quality has the highest frequency (~390), followed by premium (~275), very good (~225), then jumping down to good (~80), then fair (~40).

```r
hist(diamonds_sample$y,
     main = "Histogram of Diamonds Sample Width (y)",
       xlab = "Diamonds Sample Width (mm)",
       ylab = "Frequency")
```



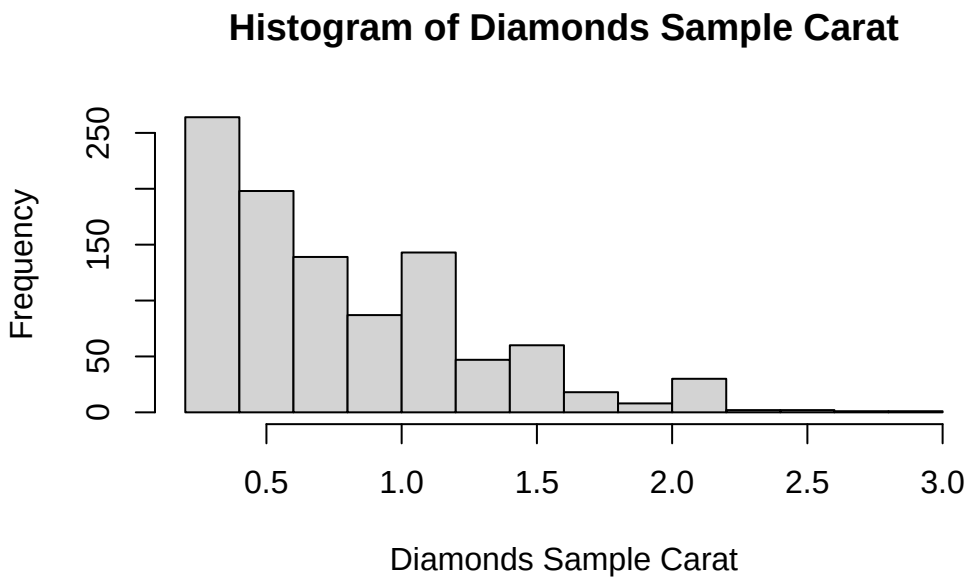

The histogram of y (width) seems to appear right-skewed with similar frequency across values y=4.5 until y=7.

```
hist(diamonds_sample$price,
     main = "Histogram of Diamonds Sample Price",
       xlab = "Diamonds Sample Price ($)",
       ylab = "Frequency")
```



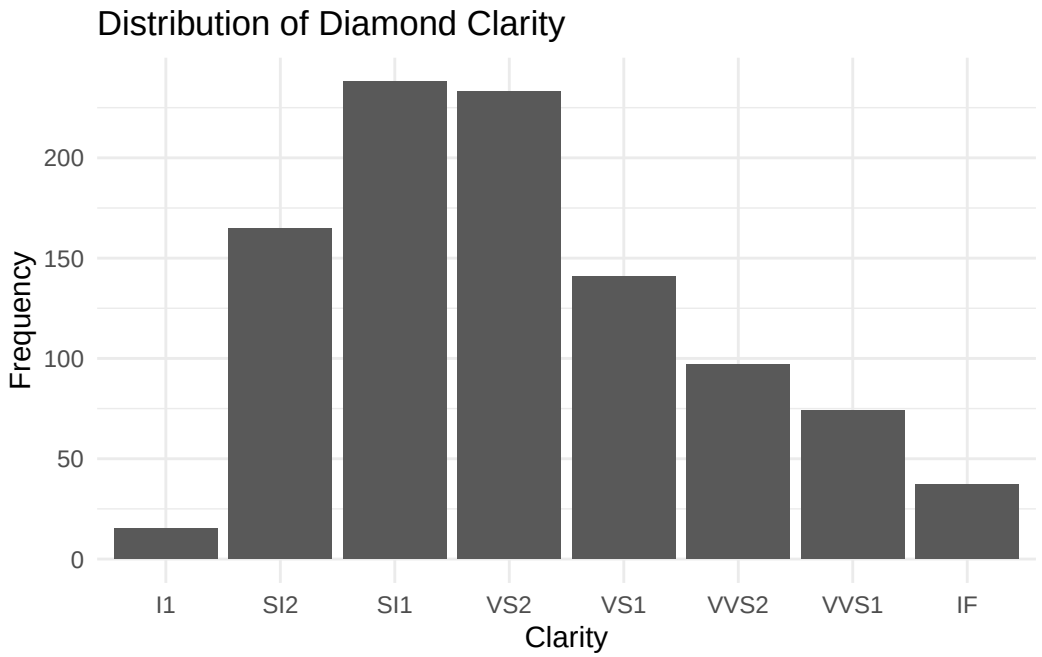

Price is heavily right-skewed.

```
hist(diamonds_sample$carat,
     main = "Histogram of Diamonds Sample Carat",
       xlab = "Diamonds Sample Carat",
       ylab = "Frequency")
```



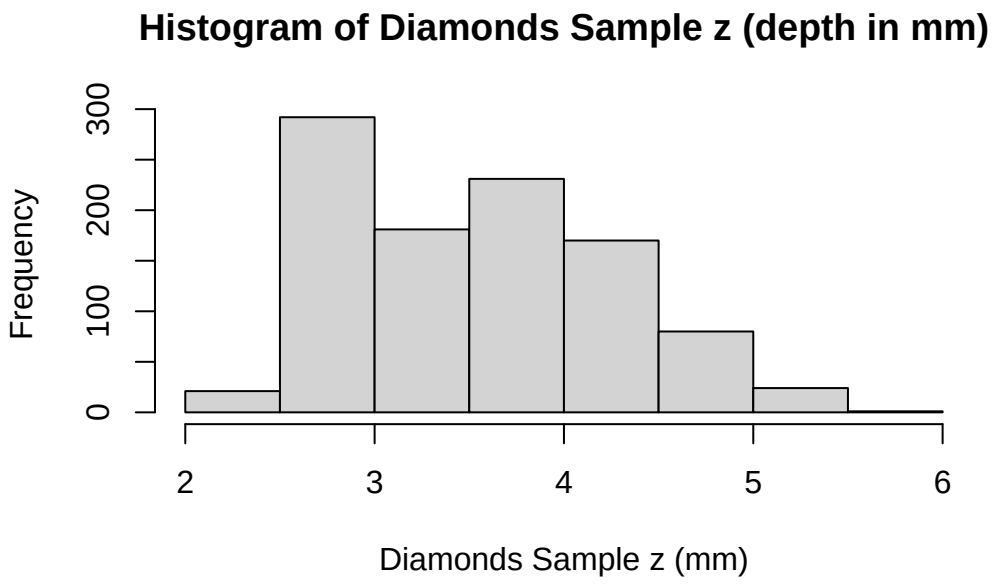

Carats is heavily right-skewed.

```
ggplot(diamonds_sample, aes(x = clarity)) +
  geom_bar() +
  labs(title = "Distribution of Diamond Clarity",
       x = "Clarity",
       y = "Frequency") +
  theme_minimal()
```



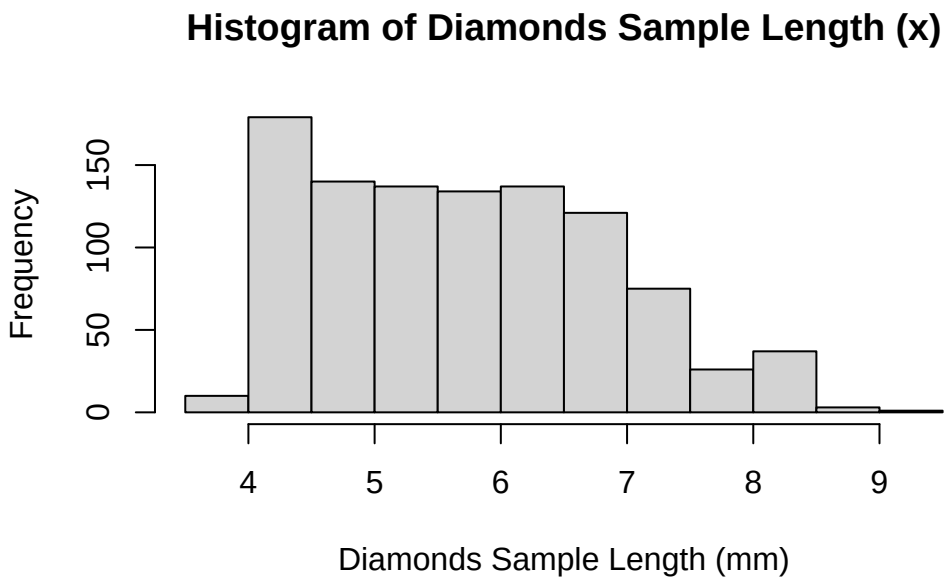

SI1 (~240) and VS2 (~230) have the highest frequencies, then SI2 (~155) and VS1 (~130), then VVS2 (~98), VVS1 (~75), IF (~35), and I1 (~20).

```
hist(diamonds_sample$z,
     main = "Histogram of Diamonds Sample z (depth in mm)",
       xlab = "Diamonds Sample z (mm)",
       ylab = "Frequency")
```



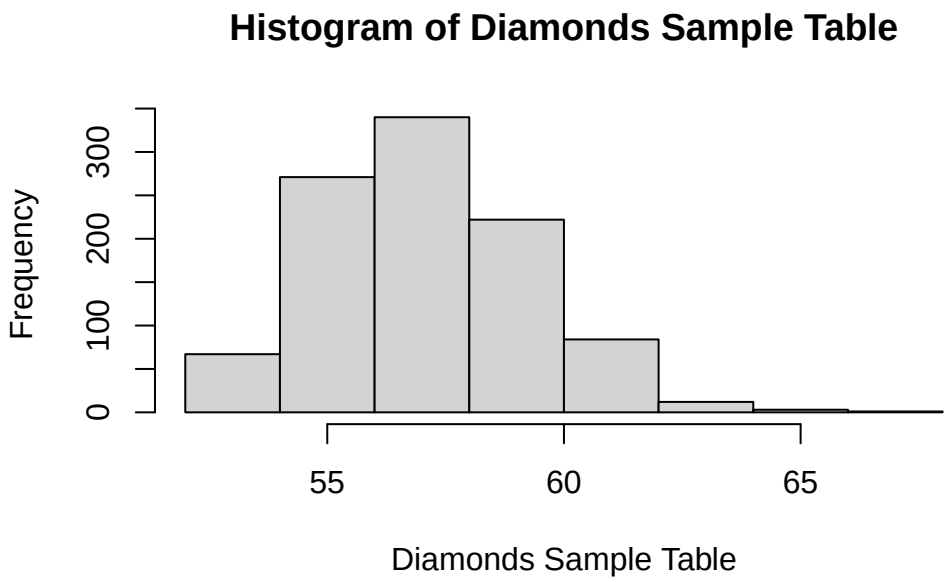

z (depth in mm) appears to be right-skewed

```
hist(diamonds_sample$x,
     main = "Histogram of Diamonds Sample Length (x)",
       xlab = "Diamonds Sample Length (mm)",
       ylab = "Frequency")
```



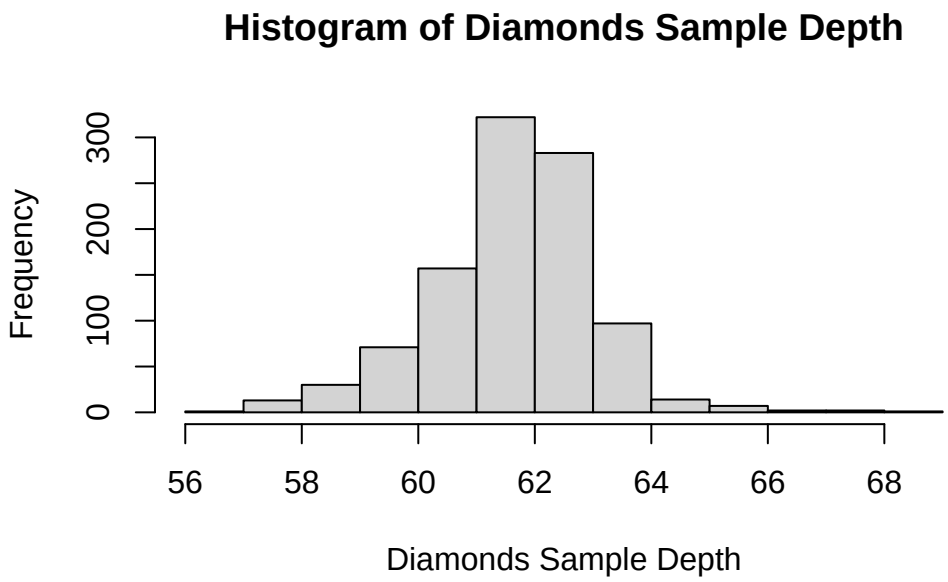

Length (x) has a similar distribution as width (y).

```
hist(diamonds_sample$table,
     main = "Histogram of Diamonds Sample Table",
       xlab = "Diamonds Sample Table",
       ylab = "Frequency")
```



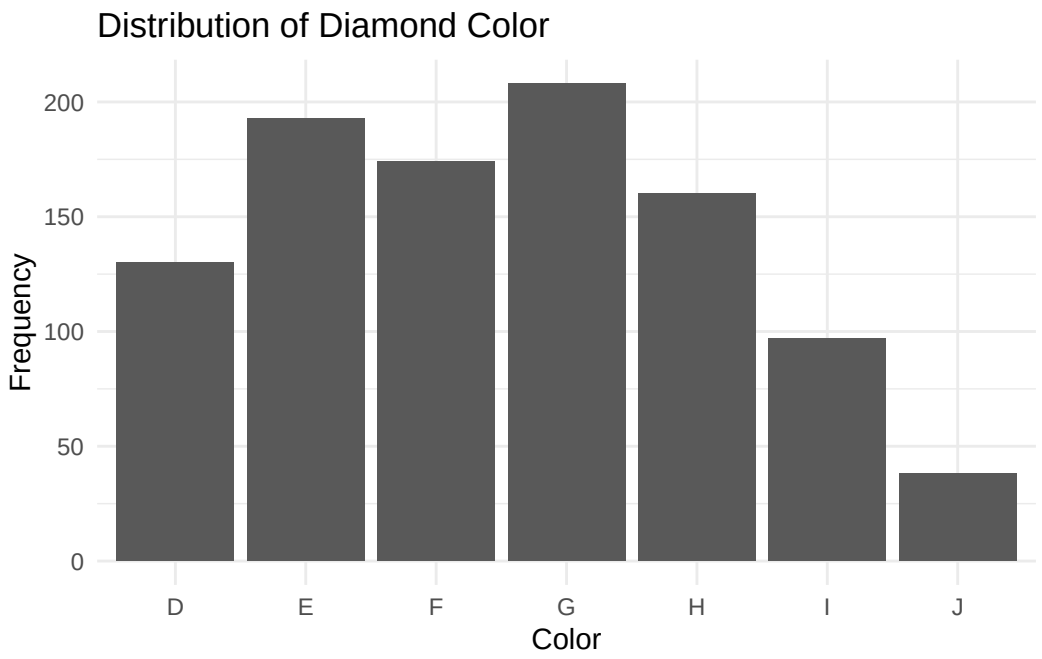

The histogram for table appears relatively normally distributed with the mean at around 57

```
hist(diamonds_sample$depth,
     main = "Histogram of Diamonds Sample Depth",
       xlab = "Diamonds Sample Depth",
       ylab = "Frequency")
```




The depth appears to be relatively normally distributed with the mean at around 61-63

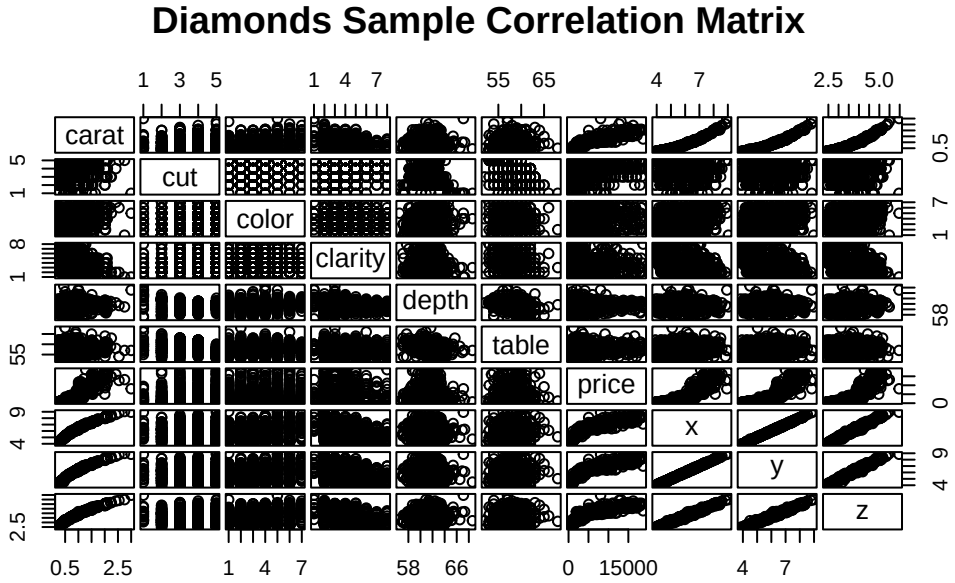
```
ggplot(diamonds_sample, aes(x = color)) +
  geom_bar() +
  labs(title = "Distribution of Diamond Color",
       x = "Color",
       y = "Frequency") +
  theme_minimal()
```




All the colors seem to be relatively similar in frequency except for J. More specifically, the highest frequency is G (~205), followed by E (~190), F (~175), H (~165), D (~130), I (~96), and lastly J (~35).

3. Determine if there is any correlation between these variables. 2 points

```
require(graphics)
pairs(diamonds_sample, main = "Diamonds Sample Correlation Matrix", gap = 1/4)
```




**There does appear to be a correlation between carat, length (x), width (y), depth (z), and price. Cut, color, clarity, depth, and table do not seem to have any correlation with either each other or any other variables.**

4. Run the multiple linear regression model using all these variables and observe the summary statistics. (DO NOT EXPLAIN HYPOTHESIS TESTING OR ANYTHING ELSE) 2 points

```
model <- lm(price ~ ., data=diamonds_sample)
summary(model)
```

```
Call:
lm(formula = price ~ ., data = diamonds_sample)

Residuals:
     Min       1Q   Median       3Q      Max
-10086.0   -560.6   -163.4    347.2   8828.7

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept) -32214.375   8977.582  -3.588 0.000349 ***
carat        12191.002    392.832  31.034  < 2e-16 ***
```

```
cut.L          596.000    183.505   3.248 0.001202 **
cut.Q         -329.348    150.496  -2.188 0.028876 *
cut.C          121.384    128.711   0.943 0.345879
cut^4         -234.837     95.117  -2.469 0.013722 *
color.L      -2091.275    139.631 -14.977  < 2e-16 ***
color.Q       -988.457    126.999  -7.783 1.80e-14 ***
color.C       -141.100    114.493  -1.232 0.218102
color^4       -153.274    102.452  -1.496 0.134962
color^5        -28.859     94.107  -0.307 0.759163
color^6         -4.544     85.894  -0.053 0.957818
clarity.L     4488.419    219.492  20.449  < 2e-16 ***
clarity.Q    -2459.064    203.730 -12.070  < 2e-16 ***
clarity.C     1386.250    172.428   8.040 2.59e-15 ***
clarity^4     -556.235    138.541  -4.015 6.40e-05 ***
clarity^5      341.111    113.652   3.001 0.002756 **
clarity^6      -61.615    100.537  -0.613 0.540112
clarity^7       46.182     90.499   0.510 0.609951
depth          580.447    142.130   4.084 4.79e-05 ***
table          -38.677     22.249  -1.738 0.082471 .
x              568.420   1051.905   0.540 0.589064
y             4710.153   1114.107   4.228 2.58e-05 ***
z           -10797.327   2376.493  -4.543 6.23e-06 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1124 on 976 degrees of freedom
Multiple R-squared:  0.9234,    Adjusted R-squared:  0.9216
F-statistic: 511.5 on 23 and 976 DF,  p-value: < 2.2e-16
```

5. Comment on anything of interest that occurred in this part. Were the data approximately what you expected, or did some of the results surprise you? 2 points

**The data was for the most part as expected because almost each predictor has a significant impact on the price of the diamond, which makes sense. It is strange that length was not significant in the model since it had high correlation with price as well as width and depth. It is also a little surprising how table seemed insignificant because I would expect that diamond shapes and ratios would impact the price of the diamond.**

## Part - 2

SIMPLE LINEAR REGRESSION

PART I continuation...

1. Start with one predictor and one response from the variables in Part I. For instance, you can start with the predictor 'carat' and the response 'price', and conduct a simple linear regression analysis on it. 2 points

```
# Predictor: x (length in mm), response: price
simple_model <- lm(price ~ x, data = diamonds_sample)
```

2. Run the model and examine the summary statistics, interpreting everything (hypothesis testing, R^2adj as discussed in class, confidence interval, prediction interval, plot, etc.). 7 points

```
summary(simple_model)
```

```
Call:
lm(formula = price ~ x, data = diamonds_sample)

Residuals:
    Min      1Q  Median      3Q     Max
-6389.3 -1223.8  -196.5   912.5 10474.1

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -14117.34     304.96  -46.29   <2e-16 ***
x            3151.38      52.36   60.18   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 1866 on 998 degrees of freedom
Multiple R-squared:  0.784, Adjusted R-squared:  0.7838
F-statistic:  3622 on 1 and 998 DF,  p-value: < 2.2e-16
```

```
# Getting the confidence interval of the estimate for the slope of x
confint(simple_model)
```

```
                2.5 %    97.5 %
(Intercept) -14715.779 -13518.90
x            3048.623   3254.13
```

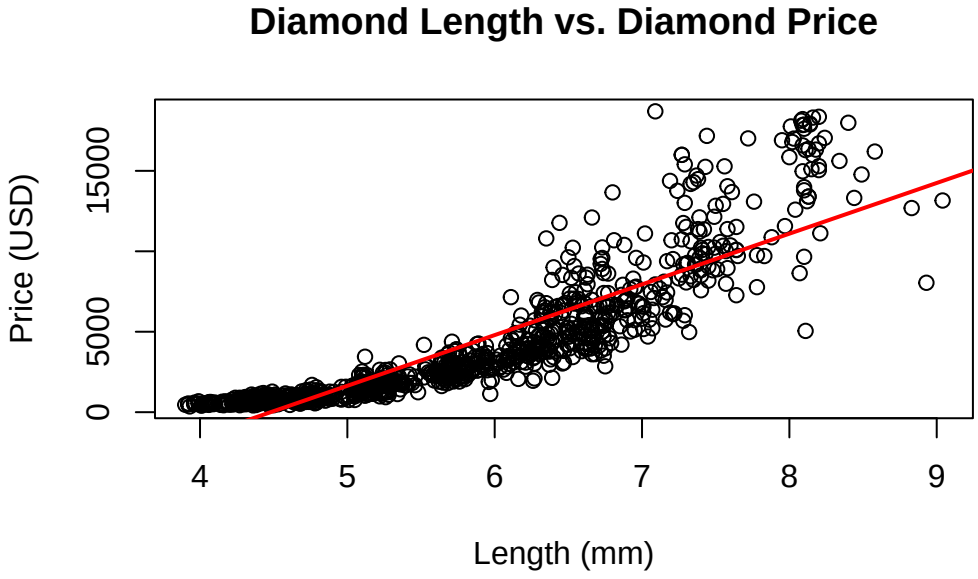
```r
# Getting the mean confidence interval of price at x = 6
x_0 <- data.frame(x = c(6))
predict(simple_model, newdata = x_0, interval = "confidence")
```

```
       fit      lwr      upr
1 4790.918 4671.433 4910.404
```

```r
# Prediction interval of price at x = 6
predict(simple_model, newdata = x_0, interval = "prediction")
```

```
       fit      lwr    upr
1 4790.918 1126.636 8455.2
```

```r
# Plotting the model
plot(price ~ x, data = diamonds_sample, xlab = "Length (mm)", ylab = "Price (USD)", main = "
abline(simple_model, col = 'red', lwd = 2)
```




**H0: The length of diamonds in millimeters does not have a significant effect on the price.**

**H1: The length of diamonds in millimeters does have a significant effect on the price.**

**Intercept:**

- **When the length of the diamond in 0mm, the estimated price is -$14,117.34. This intercept does not make much sense in the context of the data but is significant in the model because the p-value is less than alpha=0.05.**
- **The standard error is 304.96, so the average deviation of the intercept estimate from its true population value is 304.96. This shows a great deviation around the intercept estimate.**
- **The t-value is -46.29, which means that the intercept coefficient is -46.29 standard errors from 0. This means the intercept coefficient is 46.29 standard errors to the left of 0.**

**Coefficient:**

- **The point estimate for the coefficient for length (x) is 3,151.38, which means that with each increase in 1 millimeter in diamond length, the estimated price increases by $3,151.38.**
- **Regressing diamond price based on length in millimeters, we get that the length (x) is a significant factor. Under the t-test for significance, the p-value of length (x) being insignificant is much less than alpha=0.05 which implies we should reject the null hypothesis because we have significant evidence that length of diamond (x) is a significant factor on price.**
- **The standard error is 52.36, which means that the average deviation of diamond length estimate from its true population value is 52.36. This shows that the the estimate is relatively precise.**
- **The t-value is 60.18, which means that the diamonds length is 60.18 standard errors from 0. This shows strong evidence that diamond length significant affects price.**

**The R-squared score is 0.784 which means that in the model, 78.4% of the variance in prices is explained by the length of the diamond (x). This is a high R-squared score, so length is a great predictor for price. In a simple linear regression model, R-squared adjusted does not matter compared to simple R-squared since there is only one factor.**

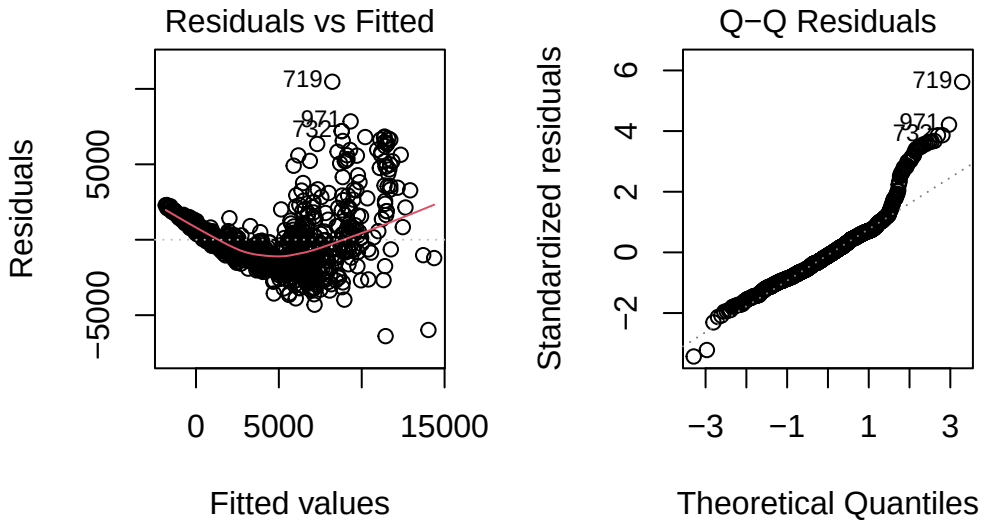
**The true estimate for the coefficient for length is likely within the 95% confidence interval (3,048.623, 3,254.13). For a given length of 6 millimeters, the 95% confidence interval for the mean value of price in dollars is (4,671.433, 4,910.404)**

**Running a prediction interval on the model for a diamond with length 6 millimeters, the 95% prediction interval is (1,126.636, 8,455.2). Plotting diamond length (x)**

**against diamond price, there does seem to be a linear relationship, though the assumption on constant variance might be violated in this current model.**

3. Test the assumptions and apply any necessary transformations to the response variable y or the predictor. 7 points

```
# Show residuals vs fitted plot to test constant variance, Q-Q plot to test normality
par(mfrow = c(1, 2))
plot(simple_model, which = c(1, 2))
```




```
# If max/min > 10 in price, we can use the log transformation
max(diamonds_sample$price, na.rm = TRUE) / min(diamonds_sample$price, na.rm = TRUE)
```

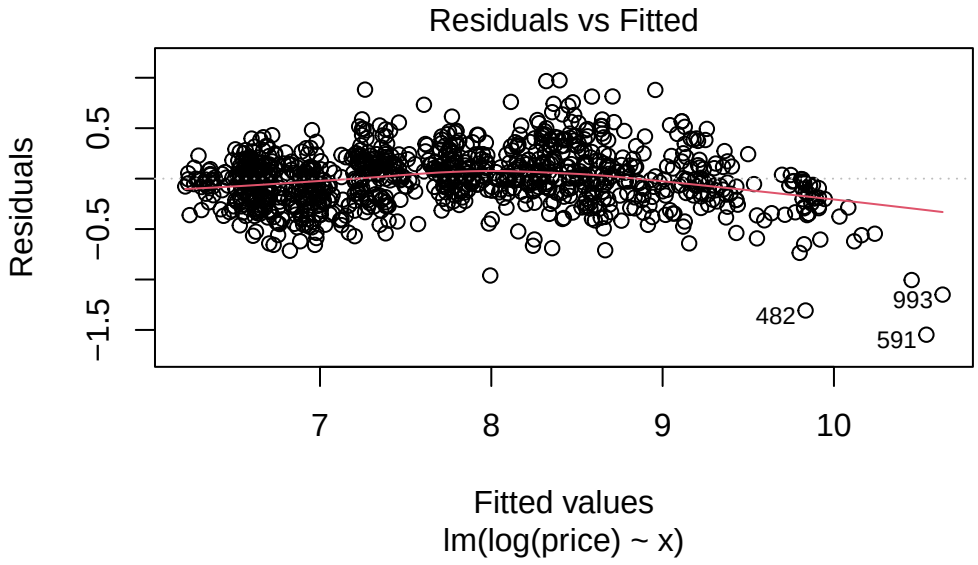
```
[1] 52.38095
```

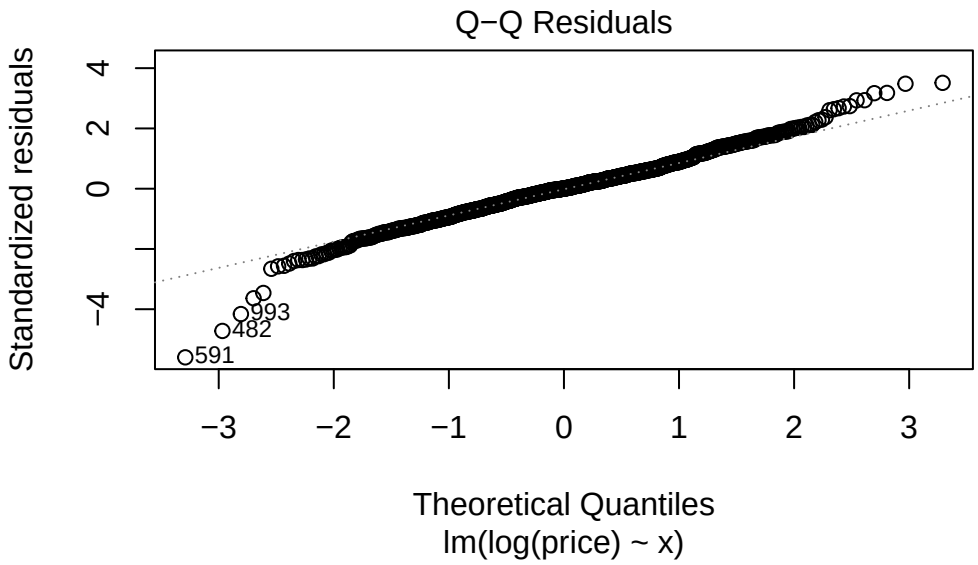
```
# Test assumption that the expected value of the error is 0
mean(residuals(simple_model))
```

```
[1] 1.320055e-14
```

**The model appears to have a funnel shape in the residuals vs fitted plot, which appears to violate the assumption that the variance is constant. In a simple linear regression model, the residuals vs fitted plot serves the same purpose as the residuals vs predictor plot. The Q-Q plot has most points close to the normal line but with some big tails, so the distribution may not be normal but can be transformed into one. We can fix this by applying a log transformation to the price. It does appear that the mean of the residuals is close to 0 so that assumption is not violated.**

```
# Perform a log transformation on the model
log_model <- lm(log(price) ~ x, data = diamonds_sample)
plot(log_model, which = c(1, 2))
```






4. Call the summary function on the transformed variables, observe the summary, and note any changes. 2 points

```
summary(log_model)
```

```
Call:
lm(formula = log(price) ~ x, data = diamonds_sample)

Residuals:
     Min       1Q   Median       3Q      Max
-1.54707 -0.16763  0.00029  0.15793  0.97487

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 2.859329   0.045372   63.02   <2e-16 ***
x           0.860070   0.007791  110.40   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.2777 on 998 degrees of freedom
Multiple R-squared:  0.9243,    Adjusted R-squared:  0.9242
F-statistic: 1.219e+04 on 1 and 998 DF,  p-value: < 2.2e-16
```

**The estimates and standard error are significantly smaller mainly due to the log transformation. The t-values are very different in the log model but the significance test still points towards the length (x) being a significant predictor of price. This model seems to follow the the assumptions of a linear model. The residuals vs fitted plot shows no discernible pattern between residuals and fitted values with mean 0 which indicates the variance is constant. The Q-Q plot has the points following a much straighter line than the other model which suggests this model does not violate the assumption on normality of the error. The log transformation does not change the expectation of the error, so that assumption is still satisfied.**

5. Add other variables to the model and assess if the model improves. For step 5, run the code in the background and include all interpretations in the file. For instance, if adding depth to the simple linear regression model (carat and price) increases the adjusted R^2, include it in the model; if it decreases, exclude it. Do not include the code for step 5 in the submitted file; only write the conclusions. 6 points

**For the simple linear regression model with length as the predictor with the log transformation, we get an R-squared score of 0.9243. We will test adding one predictor for every remaining predictor to create a multiple linear regression model with two predictors. Adding carat to the simple linear regression model we get an increased adjusted R-squared score of 0.9319. Adding cut to the simple linear regression model we get a slightly increased adjusted R-squared score of 0.9269. If we instead add color, the adjusted R-squared score goes up to 0.9354. If we add clarity, the adjusted R-squared score goes up to 0.954. If we add y, the adjusted R-squared score increases to 0.9263. If we add z, the adjusted R-squared score increases ever so slightly to 0.9249. If we add depth, the adjusted R-squared score increases slightly to 0.9246. If we add table, the adjusted R-squared score increases to 0.9254. The predictor that adds most to the adjusted R-squared score to the simple linear regression model is clarity.**

6. Comment on anything of interest that occurred while doing this part. 2 points

**Adding any of the other predictors to our log simple linear regression model increased the adjusted R-squared score. It is also interesting that clarity was the predictor that added the most to adjusted R-squared when paired with length.**

## Part -3

PART II continuation...

1. In class we saw different techniques and criterion to find best model. You can use any method and technique you prefer (e.g., backward elimination using AIC or stepwise regression

using AIC or backward elimination using BIC criterion) to find the best model and document your observations. 10 points

The number of observations is not too large so we can do stepwise regression using AIC

```
full_model <- lm(log(price) ~ ., data = diamonds_sample)
null_model <- lm(log(price) ~ 1, data = diamonds_sample)
AIC_model <- step(null_model,
                  direction = "both",
                  scope = formula(full_model))
```

```
Start:  AIC=18.51
log(price) ~ 1

          Df Sum of Sq     RSS      AIC
+ y        1    941.83   74.81 -2588.80
+ x        1    939.70   76.95 -2560.66
+ z        1    932.67   83.97 -2473.26
+ carat    1    866.32  150.32 -1890.98
+ clarity  7     83.87  932.78   -53.59
+ table    1     26.69  989.96    -6.09
+ color    6     28.41  988.24     2.17
+ cut      4     10.01 1006.63    16.61
<none>                 1016.64    18.51
+ depth    1      0.05 1016.60    20.46

Step:  AIC=-2588.8
log(price) ~ y

          Df Sum of Sq     RSS      AIC
+ clarity  7     28.16   46.65 -3047.01
+ color    6     11.92   62.89 -2750.31
+ carat    1      7.77   67.04 -2696.47
+ cut      4      2.05   72.76 -2608.54
+ table    1      1.07   73.74 -2601.25
+ depth    1      0.63   74.18 -2595.31
+ z        1      0.48   74.33 -2593.27
<none>                   74.81 -2588.80
+ x        1      0.01   74.80 -2586.92
- y        1    941.83 1016.64    18.51

Step:  AIC=-3047.01
log(price) ~ y + clarity
```

```
          Df Sum of Sq    RSS     AIC
+ color    6     16.70  29.96 -3478.0
+ carat    1      7.50  39.16 -3220.2
+ z        1      2.37  44.28 -3097.1
+ depth    1      2.22  44.44 -3093.7
+ x        1      0.65  46.01 -3059.0
+ table    1      0.37  46.28 -3053.0
<none>                  46.65 -3047.0
+ cut      4      0.31  46.35 -3045.6
- clarity  7     28.16  74.81 -2588.8
- y        1    886.13 932.78   -53.6

Step:  AIC=-3477.99
log(price) ~ y + clarity + color

          Df Sum of Sq    RSS     AIC
+ carat    1      4.74  25.22 -3648.0
+ z        1      3.46  26.50 -3598.6
+ depth    1      3.29  26.67 -3592.2
+ x        1      0.79  29.17 -3502.7
+ table    1      0.41  29.55 -3489.8
<none>                  29.96 -3478.0
+ cut      4      0.15  29.81 -3475.0
- color    6     16.70  46.65 -3047.0
- clarity  7     32.94  62.89 -2750.3
- y        1    875.87 905.83   -70.9

Step:  AIC=-3648.04
log(price) ~ y + clarity + color + carat

          Df Sum of Sq     RSS     AIC
+ z        1     6.995  18.227 -3970.8
+ depth    1     6.021  19.201 -3918.8
+ x        1     1.785  23.437 -3719.4
+ table    1     0.453  24.769 -3664.2
<none>                  25.222 -3648.0
+ cut      4     0.187  25.035 -3647.5
- carat    1     4.735  29.957 -3478.0
- color    6    13.934  39.156 -3220.2
- clarity  7    32.322  57.544 -2837.2
- y        1    83.244 108.466 -2191.3
```

```
Step:  AIC=-3970.84
log(price) ~ y + clarity + color + carat + z

          Df Sum of Sq    RSS     AIC
+ cut      4     0.817 17.411 -4008.7
+ x        1     0.652 17.575 -4005.3
+ depth    1     0.156 18.071 -3977.4
<none>                 18.227 -3970.8
+ table    1     0.002 18.225 -3969.0
- z        1     6.995 25.222 -3648.0
- carat    1     8.272 26.499 -3598.6
- y        1     9.428 27.655 -3556.0
- color    6    14.627 32.854 -3393.7
- clarity  7    36.461 54.688 -2886.1

Step:  AIC=-4008.68
log(price) ~ y + clarity + color + carat + z + cut

          Df Sum of Sq    RSS     AIC
+ x        1     0.495 16.915 -4035.5
+ table    1     0.110 17.301 -4013.0
+ depth    1     0.075 17.336 -4011.0
<none>                 17.411 -4008.7
- cut      4     0.817 18.227 -3970.8
- z        1     7.624 25.035 -3647.5
- y        1     7.990 25.401 -3633.0
- carat    1     8.636 26.046 -3607.9
- color    6    14.845 32.256 -3404.1
- clarity  7    34.380 51.790 -2932.6

Step:  AIC=-4035.54
log(price) ~ y + clarity + color + carat + z + cut + x

          Df Sum of Sq    RSS     AIC
+ table    1     0.085 16.830 -4038.6
+ depth    1     0.040 16.875 -4035.9
<none>                 16.915 -4035.5
- y        1     0.111 17.026 -4031.0
- x        1     0.495 17.411 -4008.7
- cut      4     0.660 17.575 -4005.3
- z        1     6.256 23.171 -3722.8
- carat    1     8.943 25.858 -3613.1
- color    6    14.628 31.543 -3424.4
```

```
- clarity  7    34.874 51.789 -2930.6

Step:  AIC=-4038.59
log(price) ~ y + clarity + color + carat + z + cut + x + table

          Df Sum of Sq    RSS     AIC
+ depth    1     0.055 16.775 -4039.9
<none>                 16.830 -4038.6
- table    1     0.085 16.915 -4035.5
- y        1     0.098 16.928 -4034.8
- x        1     0.471 17.301 -4013.0
- cut      4     0.743 17.573 -4003.4
- z        1     5.995 22.825 -3735.9
- carat    1     9.018 25.848 -3611.5
- color    6    14.567 31.397 -3427.0
- clarity  7    34.950 51.780 -2928.7

Step:  AIC=-4039.86
log(price) ~ y + clarity + color + carat + z + cut + x + table +
    depth

          Df Sum of Sq    RSS     AIC
<none>                 16.775 -4039.9
- depth    1     0.055 16.830 -4038.6
- z        1     0.087 16.862 -4036.7
- table    1     0.100 16.875 -4035.9
- y        1     0.150 16.925 -4032.9
- x        1     0.471 17.246 -4014.2
- cut      4     0.766 17.541 -4003.2
- carat    1     8.847 25.622 -3618.3
- color    6    14.615 31.391 -3425.2
- clarity  7    34.966 51.741 -2927.5
```

**Running the AIC method, it seems that no variable in the model after the log transformation is irrelevant. The model with the lowest AIC from the full model is:** `log(price) ~ y + clarity + color + carat + z + cut + x + table + depth`

```
summary(AIC_model)
```

```
Call:
lm(formula = log(price) ~ y + clarity + color + carat + z + cut +
```

```
    x + table + depth, data = diamonds_sample)

Residuals:
     Min       1Q   Median       3Q      Max
-0.48791 -0.08327 -0.00516  0.08694  0.37392

Coefficients:
             Estimate Std. Error t value Pr(>|t|)
(Intercept) -1.767455   1.047267  -1.688  0.09179 .
y            0.384075   0.129965   2.955  0.00320 **
clarity.L    0.954463   0.025604  37.277  < 2e-16 ***
clarity.Q   -0.304511   0.023766 -12.813  < 2e-16 ***
clarity.C    0.152308   0.020114   7.572 8.50e-14 ***
clarity^4   -0.083708   0.016161  -5.180 2.70e-07 ***
clarity^5    0.020504   0.013258   1.547  0.12229
clarity^6   -0.013670   0.011728  -1.166  0.24407
clarity^7    0.025909   0.010557   2.454  0.01430 *
color.L     -0.450232   0.016289 -27.641  < 2e-16 ***
color.Q     -0.134690   0.014815  -9.092  < 2e-16 ***
color.C     -0.024145   0.013356  -1.808  0.07094 .
color^4     -0.029383   0.011951  -2.459  0.01412 *
color^5     -0.004380   0.010978  -0.399  0.68996
color^6     -0.003192   0.010020  -0.319  0.75015
carat       -1.039664   0.045825 -22.688  < 2e-16 ***
z            0.624571   0.277226   2.253  0.02449 *
cut.L        0.141510   0.021407   6.611 6.29e-11 ***
cut.Q       -0.072482   0.017556  -4.129 3.96e-05 ***
cut.C        0.042367   0.015015   2.822  0.00487 **
cut^4       -0.012270   0.011096  -1.106  0.26909
x            0.642544   0.122709   5.236 2.01e-07 ***
table        0.006269   0.002595   2.415  0.01590 *
depth        0.029636   0.016580   1.787  0.07417 .
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.1311 on 976 degrees of freedom
Multiple R-squared:  0.9835,    Adjusted R-squared:  0.9831
F-statistic:  2529 on 23 and 976 DF,  p-value: < 2.2e-16
```

2. Detect multicollinearity among the variables using the variance inflation factor (VIF). 4 points

```
library(car)

vif(AIC_model)
```

```
               GVIF Df GVIF^(1/(2*Df))
y       1228.849974  1       35.054956
clarity    1.717696  7        1.039398
color      1.300812  6        1.022158
carat     27.744092  1        5.267266
z       2155.442858  1       46.426747
cut        2.506403  4        1.121712
x       1112.895009  1       33.360081
table      1.878684  1        1.370651
depth     30.836732  1        5.553083
```

**Because our VIF is greater than 5 or 10 for multiple variables, this shows that the associated regression coefficients are poorly estimated due to multicollinearity. Therefore, we should try removing each of them from the model to see their effects. We'll try removing x, y, and z independently to see which leaves us with the least VIF:**

```
remove_z_model <-
  lm(log(price) ~
       y + clarity + color + carat +
       cut + x + table + depth,
     data = diamonds_sample)

print("Removing z:")
```

```
[1] "Removing z:"
```

```
vif(remove_z_model)
```

```
              GVIF Df GVIF^(1/(2*Df))
y       642.288983  1       25.343421
clarity   1.670017  7        1.037310
color     1.284148  6        1.021060
carat    27.207561  1        5.216087
cut       2.462099  4        1.119214
x       658.130981  1       25.654064
```

```
table     1.875945  1        1.369652
depth     1.413701  1        1.188991
```

```
remove_y_model <-
  lm(log(price) ~
       z + clarity + color + carat +
       cut + x + table + depth,
     data = diamonds_sample)

print("Removing y:")
```

```
[1] "Removing y:"
```

```
vif(remove_y_model)
```

```
               GVIF Df GVIF^(1/(2*Df))
z       1126.595785  1       33.564800
clarity    1.666567  7        1.037157
color      1.292014  6        1.021580
carat     27.659556  1        5.259235
cut        2.221515  4        1.104921
x       1101.170697  1       33.183892
table      1.876978  1        1.370029
depth     15.891088  1        3.986363
```

```
remove_x_model <-
  lm(log(price) ~
       y + clarity + color + carat +
       cut + x + table + depth,
     data = diamonds_sample)

print("Removing x:")
```

```
[1] "Removing x:"
```

```
vif(remove_z_model)
```

```
              GVIF Df GVIF^(1/(2*Df))
y       642.288983  1       25.343421
```

```
clarity   1.670017  7        1.037310
color     1.284148  6        1.021060
carat    27.207561  1        5.216087
cut       2.462099  4        1.119214
x       658.130981  1       25.654064
table     1.875945  1        1.369652
depth     1.413701  1        1.188991
```

**Removing x and z leaves us with similar VIFs so let's remove both and see where how our VIFs change:**

```
remove_xz_model <-
  lm(log(price) ~
       y + clarity + color + carat +
       cut + table + depth,
     data = diamonds_sample)

print("Removing x & z:")
```

```
[1] "Removing x & z:"
```

```
vif(remove_xz_model)
```

```
             GVIF Df GVIF^(1/(2*Df))
y       26.696787  1        5.166893
clarity  1.622558  7        1.035176
color    1.278302  6        1.020672
carat   26.656396  1        5.162983
cut      2.058881  4        1.094470
table    1.875942  1        1.369650
depth    1.404043  1        1.184923
```

**We still have two variables with $> 5$ VIF, let's now remove y since carat has a much smaller p-value in the hypothesis testing for the AIC model:**

```
remove_xyz_model <-
  lm(log(price) ~
       clarity + color + carat +
       cut + table + depth,
     data = diamonds_sample)

print("Removing x, y, & z")
```

```
[1] "Removing x, y, & z"
```

```
vif(remove_xyz_model)
```

```
            GVIF Df GVIF^(1/(2*Df))
clarity 1.535862  7        1.031124
color   1.268701  6        1.020031
carat   1.397841  1        1.182303
cut     2.024420  4        1.092163
table   1.874045  1        1.368958
depth   1.294315  1        1.137680
```

**After some removal, it seems that x, y, and z are all strongly correlated to each other and relatively correlated to carat meaning the associated regression coefficients would be poorly estimated due to multicollinearity if we kept them. Since removing them causes the VIFs of all remaining variables to be approximately equal to 1, we know that they have no correlation between each other and that no multicollinearity remains present.**

3. Give CIs for a mean predicted value and the PIs of a future predicted value for at least one combination of X's (from your final linear model). 4 points

```
# Random combination of X's from final model
x_i <- data.frame(
  clarity = "SI1",
  color = "F",
  carat = 0.67,
  cut = "Good",
  table = 60.0,
  depth = 63.5
)
```

```
# Confidence Interval:
predict(remove_xyz_model,
        newdata = x_i,
        interval = "confidence")
```

```
      fit      lwr     upr
1 7.54691 7.447711 7.64611
```

```r
# Prediction Interval:
predict(remove_xyz_model,
        newdata = x_i,
        interval = "prediction")
```

```
      fit      lwr      upr
1 7.54691 6.887605 8.206216
```

4. Summarize your report (for the final deliverable). 4 points

**The best model we found was one that that regressed the logarithm of diamond price on the variables clarity, color, carat, cut, table, and depth. We arrived at this model first by analyzing a simple linear regression model regressing diamond price on length and finding that variance was non-constant. Since the maximum price divided by the minimum price was greater than 10, we were able to remedy this by simply using log(price) as the response variable instead. Thus, we could then begin using the stepwise AIC method to select which independent variables were most important to the model in order to avoid overfitting. However, we found that the AIC method was unable to eliminate any variables so we moved on to eliminating variables via VIF to prevent multicollinearity. Through this method we eliminated the variables corresponding to diamond size (x, y, z), which left us with our final deliverable. The formula for the best model is: 'log(price) ~ clarity + color + carat + cut + table + depth'.**